



UPES ACM & ACM-W

100 Days of Code

Day: 4

UPES ACM & ACM-W 100 Days of Code Platform

Problem 1

Transpose Matrix (Leetcode #867)

<https://leetcode.com/problems/transpose-matrix/description/>

Description

A problem that teaches matrix traversal and in-place swapping techniques by converting rows into columns. Given a 2D integer matrix, return its transpose.

The transpose of a matrix is formed by converting all rows into columns and all columns into rows.

This problem is commonly asked in interviews and helps build concepts like:

- matrix traversal
- nested loops
- row-column transformation
- 2D array manipulation

which are important for efficient matrix-processing problems.

Your Task: Transpose the given matrix using an optimized approach.

Input

A 2D integer matrix.

Output

Print the transposed matrix.

Sample Input 1	Sample Output 1
matrix = [[1,2,3],[4,5,6],[7,8,9]]	[[1,4,7],[2,5,8],[3,6,9]]
Sample Input 2	Sample Output 2
matrix = [[1,2,3],[4,5,6]]	[[1,4],[2,5],[3,6]]

Explanation: For the matrix `[[1,2,3],[4,5,6],[7,8,9]]`, the element at row *i*, column *j* moves to row *j*, column *i* — so row 0 becomes column 0, row 1 becomes column 1, and so on. For a non-square matrix like `[[1,2,3],[4,5,6]]`, the result has dimensions 3×2 instead of 2×3.

Problem 2

Rotate Array by K Steps

Description

A problem that teaches array manipulation and rotation techniques by shifting elements of an array by a given number of positions.

Given an array of integers and an integer k , rotate the array to the right by k steps. Elements shifted beyond the end of the array should wrap around to the beginning.

This problem is commonly asked in interviews and helps build concepts like:

- array manipulation
- index calculation
- modular arithmetic
- optimization techniques

which are important for efficient array-processing problems.

Your Task: Rotate the given array by k steps using an optimized approach.

Input

An integer array and an integer k .

Output

Print the resulting array after rotating by k steps.

Sample Input 1	Sample Output 1
Array = [1,2,3,4,5,6,7], $k = 3$	[5,6,7,1,2,3,4]
Sample Input 2	Sample Output 2
Array = [-1,-100,3,99], $k = 2$	[3,99,-1,-100]

Explanation: For [1,2,3,4,5,6,7] with $k=3$, the last 3 elements [5,6,7] move to the front, giving [5,6,7,1,2,3,4]. Modular arithmetic handles cases where k is larger than the array length.